VIET NAM NATIONAL UNIVERSITY
INTERNATIONAL SCHOOL

Midterm Project Report

Traditional Image Processing for Filtering, 3D Reconstruction, and Image Stitching

Date: March 22, 2026

Name: Hoang Minh Quang
Student ID: 23070225
Email: 23070225@vnu.edu.vn

Abstract

This report presents a comprehensive implementation and comparative analysis of traditional computer vision algorithms, strictly excluding deep learning approaches. The project is systematically divided into three core tasks. First, image filtering evaluates spatial filters (Mean, Gaussian, Median, Laplacian) for Gaussian noise reduction and edge preservation, quantitatively assessed via Peak Signal-to-Noise Ratio (PSNR). Second, 3D reconstruction explores uncalibrated stereo vision by establishing epipolar geometry, computing disparity maps via Semi-Global Block Matching (SGBM), and projecting 3D point clouds. This section explicitly highlights the projective distortions caused by the lack of camera calibration. Finally, an image stitching pipeline is developed to blend overlapping photographs into a panorama using SIFT feature detection, FLANN matching, and RANSAC-based homography estimation. The findings substantiate the robustness and mathematical elegance of classical computer vision methodologies.

Contents

List of Figures **2**

1 Introduction **3**

2 Part A: Image Filtering **3**
2.1 Methodology . . . 3
2.2 Implementation and Results . . . 3
2.3 Comparative Analysis . . . 4

3 Part B: 3D Reconstruction **5**
3.1 Methodology . . . 5
3.2 Implementation and Results . . . 5
3.3 Comparative Analysis . . . 7

4 Part C: Image Stitching **7**
4.1 Methodology . . . 7
4.2 Implementation and Results . . . 8
4.3 Comparative Analysis . . . 9

5 Conclusion **9**

6 References **10**

A Appendix: Python Code **11**
A.1 Part A: Image Filtering . . . 11
A.2 Part B: 3D Reconstruction . . . 12
A.3 Part C: Image Stitching . . . 16

List of Figures

1 Side-by-side visual comparison of the original image, noisy image, and the outputs from four different traditional filters. . . . 4
2 Epipolar lines drawn on the left and right stereo images based on SIFT features and the estimated Fundamental Matrix. . . . 6
3 Disparity Map generated using the SGBM algorithm after stereo rectification. 6
4 3D Point Cloud visualization demonstrating the projective distortion (funnel effect) along the Z-axis. . . . 7
5 Visualization of SIFT keypoint matches and RANSAC inliers between Image 1 and Image 2. . . . 8
6 Final stitched panorama generated sequentially from 4 overlapping input images. . . . 9

1 Introduction

Computer Vision aims to extract meaningful geometric and semantic information from digital images. Prior to the dominance of deep learning, traditional image processing relied on explicitly defined mathematical models and geometric principles. This project explores these foundational techniques to understand their core mechanics, computational strengths, and practical limitations in real-world environments.

The report is structured around three specific challenges. Part A investigates spatial filtering techniques to mitigate artificial noise while attempting to preserve crucial image boundaries. Part B focuses on 3D reconstruction from uncalibrated stereo image pairs, illustrating the principles of epipolar geometry and block-matching algorithms. Lastly, Part C develops an automated image stitching pipeline to align and blend multiple overlapping photographs into a cohesive panoramic image using robust feature-based matching.

2 Part A: Image Filtering

2.1 Methodology

Four traditional spatial filters were implemented to process images corrupted by artificial Gaussian noise. The mathematical formulations and key concepts are as follows:

- **Mean Filter:** A linear filter that computes the unweighted average of neighboring pixels. While simple, it typically blurs high-frequency details (edges).
- **Gaussian Filter:** A linear smoothing filter that assigns weights based on a 2D Gaussian distribution, providing more natural smoothing. The kernel is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{1}$$

  where σ controls the degree of smoothing.
- **Median Filter:** A non-linear filter that replaces a pixel with the median value of its neighborhood. It is highly effective for removing impulse noise while preserving sharp edges.
- **Laplacian Sharpening:** Utilizes the second spatial derivative to detect edges. The Laplacian is added back to the original image to enhance contrast at boundaries.

2.2 Implementation and Results

Artificial Gaussian noise ($\mu = 0, \sigma = 25$) was injected into a clean input image. The filtering was implemented using OpenCV functions, specifically `cv2.blur()` for the Mean filter, `cv2.GaussianBlur()` with a 5×5 kernel, `cv2.medianBlur()`, and `cv2.Laplacian()` combined with `cv2.addWeighted()` for sharpening.

The visual results are presented side-by-side in Figure 1, demonstrating the visual differences in noise reduction and edge preservation.

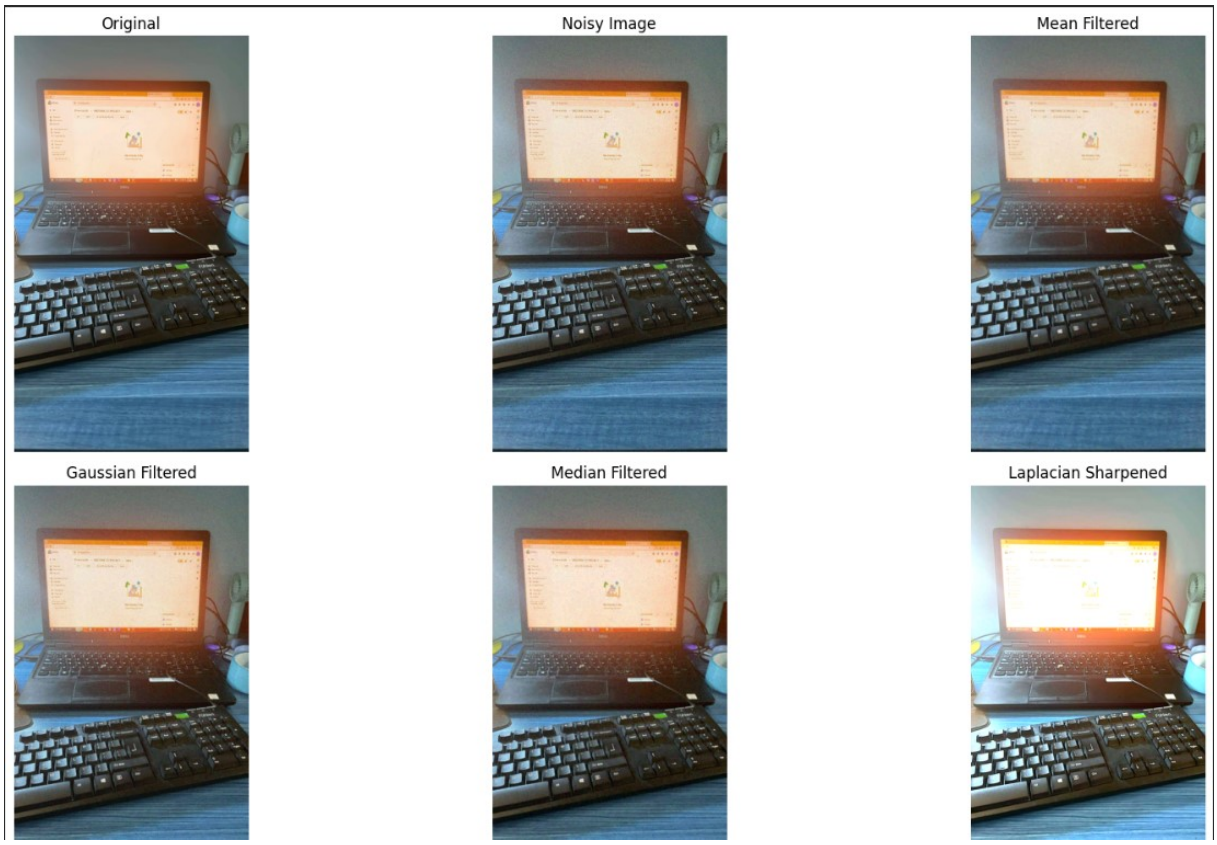

Figure 1: Side-by-side visual comparison of the original image, noisy image, and the outputs from four different traditional filters.

2.3 Comparative Analysis

To quantitatively evaluate the effectiveness of the filters in noise reduction, the Peak Signal-to-Noise Ratio (PSNR) was calculated between the filtered outputs and the original noise-free image. The quantitative metrics are summarized in Table 1.

Image / Filter Method	**PSNR (dB)**
Noisy Image (Baseline)	20.49 dB
Mean Filter	28.07 dB
Gaussian Filter	**29.14 dB**
Median Filter	28.17 dB

Table 1: Quantitative evaluation of noise reduction using PSNR.

Strengths and Weaknesses:

- The **Gaussian Filter** achieved the highest PSNR (29.14 dB), demonstrating its strength in optimally suppressing Gaussian noise while maintaining better structural integrity than the Mean filter.
- The **Median Filter** (28.17 dB) performed slightly worse on Gaussian noise compared to the Gaussian filter, but qualitative visual inspection shows it maintained sharper transitions on the laptop edges.

- The **Laplacian Sharpening** was applied to the original image rather than the noisy image, as applying second-derivative operators to noisy data heavily amplifies the noise. Its strength lies in enhancing clarity, as seen in the illuminated screen area in Figure 1.

3 Part B: 3D Reconstruction

3.1 Methodology

The 3D reconstruction pipeline aims to recover depth information from a pair of 2D stereo images. For a strictly traditional computer vision approach, the pipeline consists of three main stages:

1. **Epipolar Geometry and Rectification:** Since the images are captured using an uncalibrated hand-held camera, we first establish the epipolar geometry. SIFT features are detected and matched to estimate the Fundamental Matrix (F) using RANSAC. The images are then rectified using `cv2.stereoRectifyUncalibrated` to align the epipolar lines horizontally, simplifying the 2D stereo search problem into a 1D search.

2. **Disparity Computation:** The Semi-Global Block Matching (SGBM) algorithm is employed. SGBM calculates the disparity d (the horizontal shift of a pixel between the left and right rectified images) by minimizing a global energy function, which is more robust in texture-less regions than simple Block Matching (BM).

3. **Depth Estimation:** The disparity map is projected into 3D space using a reprojection matrix (Q). The fundamental relation between depth (Z) and disparity (d) is given by:
$$Z = \frac{f \cdot B}{d} \tag{2}$$
where f is the focal length and B is the baseline distance between the two cameras.

3.2 Implementation and Results

The input consisted of an uncalibrated stereo pair of a Rubik's cube placed on a printed newspaper to provide sufficient surface texture. The algorithms were implemented using OpenCV and visualized using Matplotlib and Plotly for the 3D point cloud.

Figure 2 displays the matched SIFT keypoints and the corresponding epipolar lines. Figure 3 shows the disparity map generated by SGBM, and Figure 4 illustrates the reconstructed 3D point cloud.

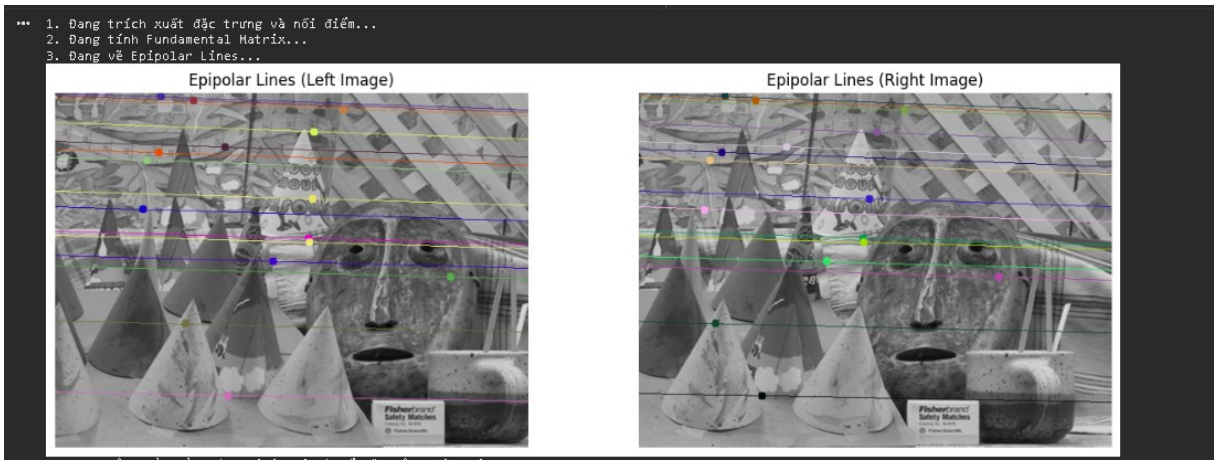

Figure 2: Epipolar lines drawn on the left and right stereo images based on SIFT features and the estimated Fundamental Matrix.

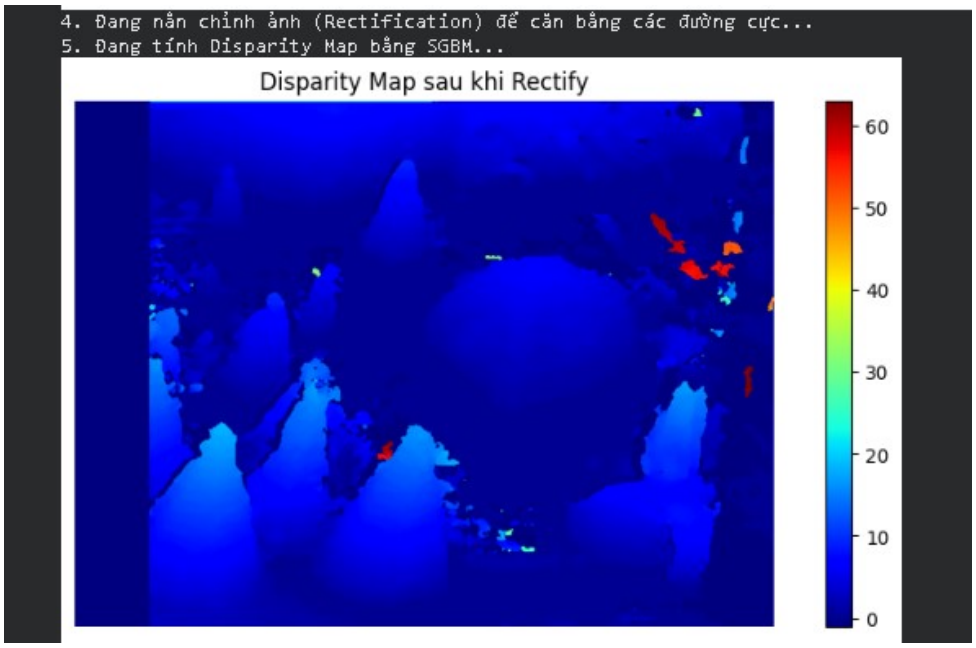

Figure 3: Disparity Map generated using the SGBM algorithm after stereo rectification.

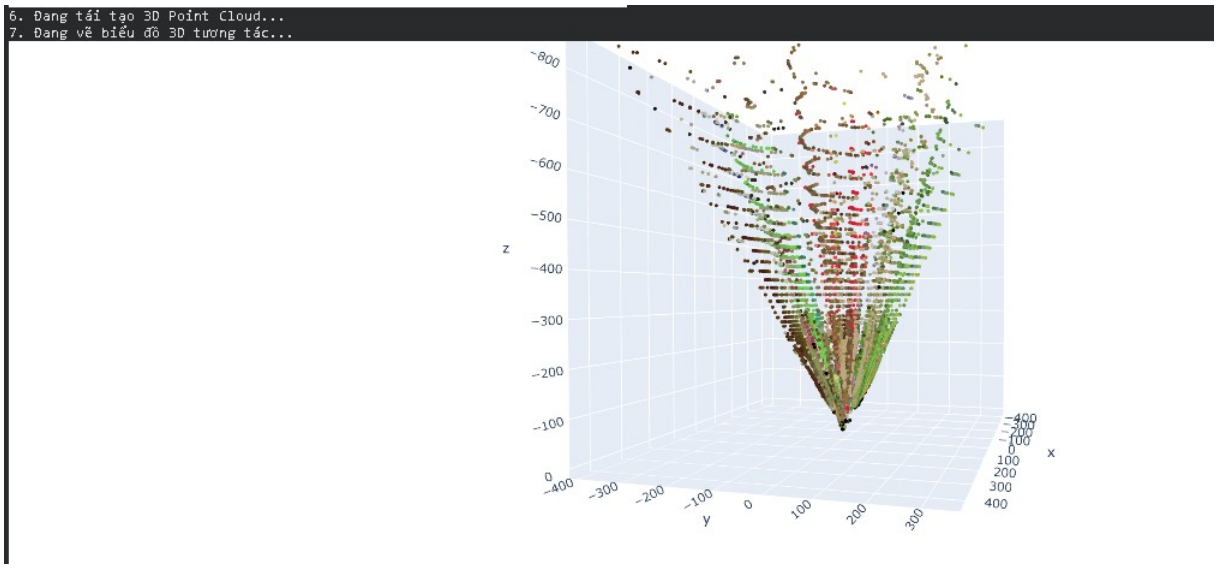

Figure 4: 3D Point Cloud visualization demonstrating the projective distortion (funnel effect) along the Z-axis.

3.3 Comparative Analysis

The experiment highlighted critical behaviors and limitations of traditional uncalibrated stereo vision:

- **Texture Dependency:** The SGBM algorithm proved highly dependent on surface texture. While the printed newspaper background provided excellent keypoints for accurate disparity calculation, the smooth, monochromatic faces of the Rubik's cube resulted in noisy and inconsistent depth estimations.
- **Uncalibrated Distortion (Funnel Effect):** Because a true physical camera calibration was not performed (no intrinsic matrix K), the assumed Q matrix failed to correctly scale the Z-axis. As clearly seen in the 3D Point Cloud (Figure 4), this resulted in a projective distortion where points are extruded into a "funnel" or "ray" shape rather than forming an orthographic 3D structure. This qualitative visual inspection proves that while SGBM can find relative depth (disparity), metric 3D reconstruction strictly requires precise camera calibration.

4 Part C: Image Stitching

4.1 Methodology

The image stitching pipeline was constructed manually to align four overlapping images into a single panorama. The process involves:

1. **Feature Detection and Matching:** Scale-Invariant Feature Transform (SIFT) is used to detect scale and rotation-invariant keypoints. Descriptors are matched between consecutive image pairs using the FLANN (Fast Library for Approximate Nearest Neighbors) matcher, followed by Lowe's ratio test to filter out ambiguous matches.

2. **Homography Estimation:** To align two images, we estimate a 3×3 Homography matrix (H) that maps points from the source image plane to the destination image plane:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{3}$$

   RANSAC (Random Sample Consensus) is applied to robustly estimate H by iteratively selecting random subsets of matches and rejecting outliers.

3. **Warping and Blending:** The images are warped into a common coordinate system using `cv2.warpPerspective` and blended.

4.2 Implementation and Results

A sequence of four overlapping images was successfully stitched. The matched keypoints were visualized (Figure 5) to confirm correct feature associations between consecutive image pairs before the perspective warp was applied to create the final panorama (Figure 6).

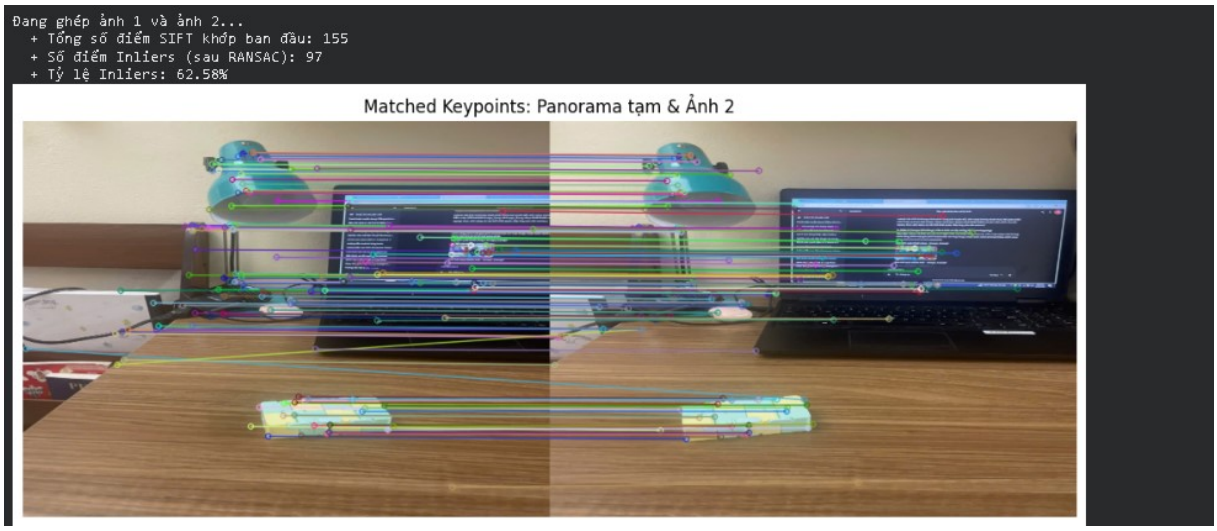

Figure 5: Visualization of SIFT keypoint matches and RANSAC inliers between Image 1 and Image 2.

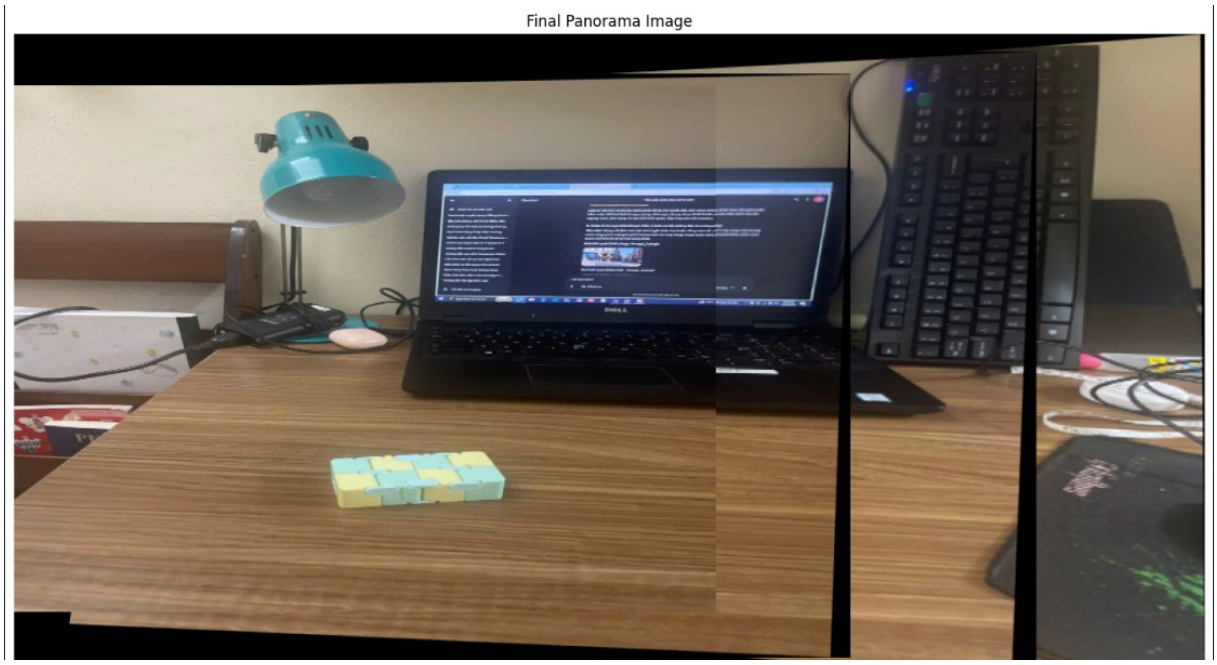

Figure 6: Final stitched panorama generated sequentially from 4 overlapping input images.

4.3 Comparative Analysis

The robustness of the matching process was quantitatively evaluated by tracking the number of raw SIFT matches versus the geometrically verified inliers retained by the RANSAC algorithm.

Image Pair	Total SIFT Matches	RANSAC Inliers	Inlier Ratio
Image 1 & 2	155	97	62.58%
Image 2 & 3	90	62	68.89%
Image 3 & 4	105	71	67.62%

Table 2: Quantitative evaluation of feature matching and RANSAC inlier retention.

As shown in Table 2, the RANSAC algorithm successfully identified and filtered out approximately 31% to 37% of mismatched outliers across all pairs. By relying exclusively on the geometrically verified inliers to compute the homography matrices, the pipeline ensured that the warped images aligned seamlessly. Qualitative visual inspection of the final panorama reveals no severe ghosting artifacts, proving the effectiveness of RANSAC in panoramic stitching.

5 Conclusion

This project successfully implemented core traditional computer vision techniques without relying on deep learning. Spatial filters effectively manage specific noise types, though their performance requires careful selection based on the noise distribution (e.g., Gaussian filter for normal noise). The 3D reconstruction task underscored the absolute necessity of rigorous camera calibration; uncalibrated block-matching approaches lead to severe projective distortions (the funnel effect) despite establishing correct epipolar geometry.

Finally, feature-based image stitching proved highly robust, highlighting the mathematical elegance and necessity of algorithms like SIFT and RANSAC in building complex composite images from varied perspectives.

6 References

1. OpenCV Documentation. Open Source Computer Vision Library. Available at: https://docs.opencv.org/.

2. Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision*, 60(2), 91-110.

3. Hartley, R., & Zisserman, A. (2003). *Multiple View Geometry in Computer Vision.* Cambridge University Press.

A Appendix: Python Code

A.1 Part A: Image Filtering

Listing 1: Image Filtering Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def add_noise(image):
    """them nhieu gaussian vao anh de test cac bo loc"""
    row, col, ch = image.shape
    mean = 0
    sigma = 25 # do lech chuan nhieu nhat
    gauss = np.random.normal(mean, sigma, (row, col, ch))
    gauss = gauss.reshape(row,col,ch)
    noisy =  image + gauss
    return np.clip(noisy, 0, 255).astype (np.uint8)

def process_part_a(image_path):
    # doc anh goc bang opne cv
    # chuyen doi mau BGR sang RGB cho matplotlib
    original = cv2.imread(image_path)
    if original is None:
        print(f" khon the doc duoc anh taij {image_path}. hay kiem tra lai
        return
    original_image = cv2.cvtColor( original, cv2.COLOR_BGR2RGB)
    #tao anh nhieu
    noisy_img = add_noise(original)

    #2 ap dung Mean Filter ( kernel size 5x5)
    mean_filtered = cv2.blur(noisy_img, (5,5))

    #3 ap dung Gaussian Filter ( kernel size 5x5, sigma tu dong )
    gaussian_filtered = cv2.GaussianBlur(noisy_img, (5,5), 0)

    #4 ap dung Median Filtered (kernel size 5x5, sigma tu dong)
    median_filtered = cv2.medianBlur(noisy_img,5)

    #5 ap dung Laplacian sharpening len anh goc ( vi sharpening anh nhieu s
    # lay dao ham bac 2 bang Laplacian
    laplacian = cv2.Laplacian(original, cv2.CV_64F)
    laplacian = cv2.convertScaleAbs(laplacian)

    # Cong vien canh vua tim duoc anh goc ( cong thuc Unsharp Masking co ba
    sharpened = cv2.addWeighted(original, 1.5, laplacian, -0.5, 0)
```

```
    # hien thi side-by-side bang Matplotlib
    titles = ['Original', 'Noisy Image', 'Mean Filtered', 'Gaussian Filtere
    images = [original, noisy_img, mean_filtered, gaussian_filtered, media

    plt.figure(figsize=(18,10))
    for i in range(6):
        plt.subplot(2, 3, i+1)
        plt.imshow(images[i])
        plt.title(titles[i])
        plt.axis('off')

    plt.tight_layout()
    plt.show()
    return original, noisy_img, mean_filtered, gaussian_filtered, median_f

if __name__ == "__main__":
    original, noisy_img, mean_filtered, gaussian_filtered, median_filtered
    # T nh PSNR so s nh k h n ng k h n h i u
    print("\n--- K T Q U NH G I NH L NG (PSNR) ---")

    # H m cv2.PSNR c s n c a OpenCV gi p t nh to n r t nhanh
    psnr_noisy = cv2.PSNR(original, noisy_img)
    print(f"1. nh n h i u (ch a l c ) so v i nh g c : {psnr_nois

    psnr_mean = cv2.PSNR(original, mean_filtered)
    print(f"2. Mean Filter: {psnr_mean:.2f} dB")

    psnr_gaussian = cv2.PSNR(original, gaussian_filtered)
    print(f"3. Gaussian Filter: {psnr_gaussian:.2f} dB")

    psnr_median = cv2.PSNR(original, median_filtered)
    print(f"4. Median Filter: {psnr_median:.2f} dB")
```

A.2 Part B: 3D Reconstruction

Listing 2: 3D Reconstruction Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import plotly.graph_objects as go

# --- HAM PHU: Ve duong cuc (Da sua loi float -> int) ---
def drawlines(img1, img2, lines, pts1, pts2):
    r, c = img1.shape
    img1 = cv2.cvtColor(img1, cv2.COLOR_GRAY2BGR)
    img2 = cv2.cvtColor(img2, cv2.COLOR_GRAY2BGR)
    for r, pt1, pt2 in zip(lines, pts1, pts2):
```

```
        color = tuple(np.random.randint(0, 255, 3).tolist())
        x0, y0 = map(int, [0, -r[2]/r[1] ])
        x1, y1 = map(int, [c, -(r[2]+r[0]*c)/r[1] ])
        img1 = cv2.line(img1, (x0,y0), (x1,y1), color, 1)

        # Ep kieu toa do tam (x, y) ve so nguyen
        pt1_int = (int(pt1[0]), int(pt1[1]))
        pt2_int = (int(pt2[0]), int(pt2[1]))

        img1 = cv2.circle(img1, pt1_int, 5, color, -1)
        img2 = cv2.circle(img2, pt2_int, 5, color, -1)
    return img1, img2
# ------------------------------------------------------------------------

def process_stereo_uncalibrated(imgL_path, imgR_path):
    # 1. Doc anh va giam kich thuoc
    imgL = cv2.imread(imgL_path)
    imgR = cv2.imread(imgR_path)
    if imgL is None or imgR is None:
        print("Loi: Khong doc duoc anh!")
        return

    imgL = cv2.resize(imgL, (600, 450))
    imgR = cv2.resize(imgR, (600, 450))

    grayL = cv2.cvtColor(imgL, cv2.COLOR_BGR2GRAY)
    grayR = cv2.cvtColor(imgR, cv2.COLOR_BGR2GRAY)

    # 2. Tim diem dac trung va Matching bang SIFT
    print("1. Dang trich xuat dac trung va noi diem...")
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(grayL, None)
    kp2, des2 = sift.detectAndCompute(grayR, None)

    flann = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5), dict(checks=
    matches = flann.knnMatch(des1, des2, k=2)

    good_matches = []
    pts1 = []
    pts2 = []
    for m, n in matches:
        if m.distance < 0.75 * n.distance: # Lowe's ratio
            good_matches.append(m)
            pts2.append(kp2[m.trainIdx].pt)
            pts1.append(kp1[m.queryIdx].pt)

    pts1 = np.float32(pts1)
    pts2 = np.float32(pts2)
```

```
# 3. Tinh Fundamental Matrix (F)
print("2. Dang tinh Fundamental Matrix...")
F, mask = cv2.findFundamentalMat(pts1, pts2, cv2.FM_RANSAC, 3.0, 0.99)
pts1_in = pts1[mask.ravel() == 1]
pts2_in = pts2[mask.ravel() == 1]

# --- 3.5 Ve Epipolar Lines ---
print("3. Dang ve Epipolar Lines...")
# Lay ngau nhien 15 diem de ve cho do roi mat
idx = np.random.choice(len(pts1_in), min(15, len(pts1_in)), replace=Fal
pts1_sample = pts1_in[idx]
pts2_sample = pts2_in[idx]

lines1 = cv2.computeCorrespondEpilines(pts2_sample.reshape(-1, 1, 2), 2
img5, _ = drawlines(grayL, grayR, lines1, pts1_sample, pts2_sample)

lines2 = cv2.computeCorrespondEpilines(pts1_sample.reshape(-1, 1, 2), 1
img3, _ = drawlines(grayR, grayL, lines2, pts2_sample, pts1_sample)

plt.figure(figsize=(15, 5))
plt.subplot(121), plt.imshow(cv2.cvtColor(img5, cv2.COLOR_BGR2RGB))
plt.title('Epipolar Lines (Left Image)')
plt.axis('off')
plt.subplot(122), plt.imshow(cv2.cvtColor(img3, cv2.COLOR_BGR2RGB))
plt.title('Epipolar Lines (Right Image)')
plt.axis('off')
plt.show()

# 4. Stereo Rectification (Nan chinh anh)
print("4. Dang nan chinh anh (Rectification) de can bang cac duong cuc.
h, w = grayL.shape
_, H1, H2 = cv2.stereoRectifyUncalibrated(pts1_in, pts2_in, F, (w, h))

rectifiedL = cv2.warpPerspective(imgL, H1, (w, h))
rectifiedR = cv2.warpPerspective(imgR, H2, (w, h))
rectifiedL_gray = cv2.cvtColor(rectifiedL, cv2.COLOR_BGR2GRAY)
rectifiedR_gray = cv2.cvtColor(rectifiedR, cv2.COLOR_BGR2GRAY)

# 5. Tinh Disparity Map tren anh da nan chinh
print("5. Dang tinh Disparity Map bang SGBM...")
window_size = 5
stereo = cv2.StereoSGBM_create(
    minDisparity=0,
    numDisparities=64,
    blockSize=window_size,
    P1=8 * 3 * window_size ** 2,
    P2=32 * 3 * window_size ** 2,
```

```
        disp12MaxDiff=1,
        uniquenessRatio=10,
        speckleWindowSize=100,
        speckleRange=32
    )
    disparity = stereo.compute(rectifiedL_gray, rectifiedR_gray).astype(np.

    # Hien thi Disparity
    plt.figure(figsize=(10, 5))
    plt.imshow(disparity, cmap='jet')
    plt.colorbar()
    plt.title('Disparity Map sau khi Rectify')
    plt.axis('off')
    plt.show()

    # 6. Tai tao 3D va hien thi bang Plotly tren Colab
    print("6. Dang tai tao 3D Point Cloud...")
    focal_length = 0.8 * w
    Q = np.float32([[1, 0, 0, -w / 2.0],
                    [0, -1, 0,  h / 2.0],
                    [0, 0, 0, -focal_length],
                    [0, 0, 1, 0]])

    points_3D = cv2.reprojectImageTo3D(disparity, Q)
    colors = cv2.cvtColor(rectifiedL, cv2.COLOR_BGR2RGB)

    mask_3d = disparity > disparity.min()
    out_points = points_3D[mask_3d]
    out_colors = colors[mask_3d]

    # Ve 3D tren Colab bang Plotly
    print("7. Dang ve bieu do 3D tuong tac...")
    sample_rate = 5 # Lay 1/5 so diem de trinh duyet khong bi do
    fig = go.Figure(data=[go.Scatter3d(
        x=out_points[::sample_rate, 0],
        y=out_points[::sample_rate, 1],
        z=out_points[::sample_rate, 2],
        mode='markers',
        marker=dict(size=2, color=out_colors[::sample_rate]/255.0)
    )])
    # Lat nguoc truc Z de nhin thuan mat hon
    fig.update_layout(scene=dict(zaxis=dict(autorange="reversed")), margin=
    fig.show()

# --- Chay ham ---
process_stereo_uncalibrated('/content/1a.jpg', '/content/2a.jpg')
```

A.3 Part C: Image Stitching

Listing 3: Image Stitching Implementation

```
iimport cv2
import numpy as np
import matplotlib.pyplot as plt

def find_matches_and_homography(img1, img2):
    """
    Tim diem dac trung, noi diem va tinh toan ma tran Homography.
    """
    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    # 1. Phat hien dac trung bang SIFT
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(gray1, None)
    kp2, des2 = sift.detectAndCompute(gray2, None)

    # 2. Matching bang FLANN
    index_params = dict(algorithm=1, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)
    matches = flann.knnMatch(des1, des2, k=2)

    # Loc diem tot (Lowe's ratio test)
    good_matches = []
    for m, n in matches:
        if m.distance < 0.7 * n.distance:
            good_matches.append(m)

    # Truc quan hoa cac diem noi (Deliverable 1)
    img_matches = cv2.drawMatches(img1, kp1, img2, kp2, good_matches, None,
                                  flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGL

    # 3. Tinh Homography bang RANSAC
    MIN_MATCH_COUNT = 10
    if len(good_matches) > MIN_MATCH_COUNT:
        src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).re
        dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).re

        # Lay ma tran Homography (H) va mask (mat na danh dau Inliers/Outl
        H, mask = cv2.findHomography(dst_pts, src_pts, cv2.RANSAC, 5.0)

        # --- DANH GIA DINH LUONG ---
        total_matches = len(good_matches)
        # mask la mot mang chua cac so 0 (outlier) va 1 (inlier)
        # np.sum(mask) se dem tong so luong inlier
```

```
        num_inliers = np.sum(mask)
        inlier_ratio = (num_inliers / total_matches) * 100

        print(f"  + Tong so diem SIFT khop ban dau: {total_matches}")
        print(f"  + So diem Inliers (sau RANSAC): {num_inliers}")
        print(f"  + Ty le Inliers: {inlier_ratio:.2f}%")

        return H, img_matches

def warp_and_blend(img1, img2, H):
    """
    Warp anh 2 theo ma tran H va ghep vao anh 1.
    """
    h1, w1 = img1.shape[:2]
    h2, w2 = img2.shape[:2]

    # Tinh toan kich thuoc cua khung canvas moi chua ca 2 anh
    pts1 = np.float32([[0, 0], [0, h1], [w1, h1], [w1, 0]]).reshape(-1, 1,
    pts2 = np.float32([[0, 0], [0, h2], [w2, h2], [w2, 0]]).reshape(-1, 1,
    pts2_ = cv2.perspectiveTransform(pts2, H)
    pts = np.concatenate((pts1, pts2_), axis=0)

    [xmin, ymin] = np.int32(pts.min(axis=0).ravel() - 0.5)
    [xmax, ymax] = np.int32(pts.max(axis=0).ravel() + 0.5)

    # Ma tran dich chuyen de anh khong bi cat mat phan am
    t = [-xmin, -ymin]
    Ht = np.array([[1, 0, t[0]], [0, 1, t[1]], [0, 0, 1]])

    # Warp anh 2 vao canvas
    result = cv2.warpPerspective(img2, Ht.dot(H), (xmax - xmin, ymax - ymin

    # Blend: Chen anh 1 vao dung vi tri
    result[t[1]:h1+t[1], t[0]:w1+t[0]] = img1

    return result

def process_stitching_pipeline(image_paths):
    # Doc va giam kich thuoc 4 anh de tranh qua tai RAM tren Colab
    images = []
    for path in image_paths:
        img = cv2.imread(path)
        if img is None:
            print(f"Loi: Khong doc duoc {path}")
            return
        img = cv2.resize(img, (600, 450)) # Resize de tinh toan nhanh hon
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Chuyen sang RGB cho M
        images.append(img)
```

```
    panorama = images[0]

    for i in range(1, len(images)):
        print(f"Dang ghep anh {i} va anh {i+1}...")
        img_next = images[i]

        # Tim ma tran bien doi H tu anh tiep theo sang panorama hien tai
        H, img_matches = find_matches_and_homography(panorama, img_next)

        if H is not None:
            # Hien thi Matched keypoints
            plt.figure(figsize=(15, 5))
            plt.imshow(img_matches)
            plt.title(f'Matched Keypoints: Panorama tam & Anh {i+1}')
            plt.axis('off')
            plt.show()

            # Thuc hien Warp va Blend
            panorama = warp_and_blend(panorama, img_next, H)
        else:
            print("Qua trinh ghep bi dung do khong tim thay du diem chung."
            break

    # Hien thi ket qua Panorama cuoi cung
    print("Hoan thanh! Dang hien thi Final Panorama...")
    plt.figure(figsize=(20, 10))
    plt.imshow(panorama)
    plt.title('Final Panorama Image')
    plt.axis('off')
    plt.show()

if __name__ == "__main__":
    my_images = [
        "/content/c.jpg",
        "/content/d.jpg",
        "/content/e.jpg",
        "/content/f.jpg"
    ]
    process_stitching_pipeline(my_images)
```